

SafeRoad AI — Technical Project Report

Deep Learning Road Scene Risk Classification, Traffic Monitoring & Full-Stack Integration

Version: 1.0.0 | Date: September 2026 | Frameworks: TensorFlow 2.18 / Keras 3, Ultralytics YOLOv8, Flask, React & Vite

Executive Summary

SafeRoad AI is an intelligent, real-time road safety and hazard mitigation platform designed to monitor traffic scenes, detect objects, evaluate environmental hazard factors, and predict accident risk severity. In this project, a complete deep learning machine learning pipeline was designed and executed from scratch without altering the existing frontend UI. Four distinct deep learning classification models (**Custom CNN Baseline**, **MobileNetV2**, **EfficientNetB0**, and **ResNet50**) were trained on the real SafeRoad multi-source dataset, tuned using Optuna Bayesian optimization, and rigorously evaluated on an identical, strictly isolated 15% unseen test set. Ultralytics YOLOv8 was integrated for real-time contextual vehicle/pedestrian detection and traffic density estimation. A high-performance Flask REST API was deployed to connect the React/Tailwind frontend, enabling live interactive predictions, 4-model benchmarking, and persistent historical analytics.

KEY HIGHLIGHT: BEST PERFORMING MODEL DETERMINED
MobileNetV2 achieved the top benchmark ranking with 80.82% Test Accuracy, 65.87% Macro-F1, 85.56% High-Risk Recall, and ultra-fast 11.3 ms/image inference latency. It serves as the default production model powering live predictions on the SafeRoad AI website.

1. Dataset Architecture & Leakage-Free Splitting

The SafeRoad AI dataset aggregates real-world road scenes across multiple challenging driving contexts: **BDD100K** (diverse highway and urban scenes under varied daylight and weather), **India Driving Dataset (IDD)** (complex unstructured traffic environments), and **Custom Intersection Video Captures**. A programmatic audit confirmed **6,949 valid, uncorrupted RGB images** across three risk categories.

Risk Class	Total Count	Proportion	Train (70%)	Val (15%)	Test (15%)	Train Weight
Safe	2,350	33.82%	1,645	353	352	0.9856
Moderate Risk	307	4.42%	215	46	46	7.5411
High Risk	4,292	61.76%	3,004	643	645	0.5397
Total / Summary	6,949	100.0%	4,864	1,042	1,043	Fixed Seed=42

Data Leakage Prevention Protocol: The 1,043 test images were strictly partitioned prior to any modeling and saved to ml/split_data.json. The test set remained completely unseen during training, validation checkpointing, and hyperparameter tuning. Class weights were calculated *strictly* on the 4,864 training samples using inverse-frequency balancing to penalize minority misclassifications.

2. Deep Learning Model Architectures & Methodologies

All four classification models share the standardized input dimensions (224 × 224 × 3 RGB) and output space (3 classes: Safe, Moderate Risk, High Risk). Data augmentation (random horizontal flip, small rotation ±10°, random zoom ±10%, brightness shift ±10%) was applied **only** during training.

Model	Backbone Type	Classification Head Architecture	Param Count	Training Protocol
Custom CNN	4-Stage Conv2D Stack (32→64→128→256 filters, 3x3, ReLU, MaxPool 2x2)	GlobalAveragePooling2D → Dense(128, ReLU) → Dropout(0.3) → Dense(3, Softmax)	421,699	Trained from scratch; Adam lr=0.001, ReduceLROnPlateau, EarlyStopping
MobileNetV2	Depthwise-Separable Convolutions, Inverted Residuals (ImageNet pre-trained)	GAP → Dense(128, ReLU) → Dropout(0.3) → Dense(3, Softmax)	2,422,339	Two-phase: Head training (frozen backbone, lr=0.001), fine-tuning top blocks (lr=0.0001)
EfficientNetB0	Compound scaling MBConv with Squeeze-and-Excitation (ImageNet pre-trained)	GAP → Dense(128, ReLU) → Dropout(0.3) → Dense(3, Softmax)	4,213,926	Feature transfer + fine-tuning upper MBConv stages with Adam lr=0.0001
ResNet50	50-layer Residual Network with Skip Connections (ImageNet pre-trained)	GAP → Dense(128, ReLU) → Dropout(0.3) → Dense(3, Softmax)	23,850,371	Rescaling internal layer (-120 mean shifted) + feature extraction fine-tuning
Ultralytics YOLOv8	YOLOv8 Nano (yolov8n.pt) object detector	Decoupled detection head: vehicle & pedestrian localization	3,157,200	Pretrained COCO weights; decoupled from risk classification; density estimation only

3. Optuna Hyperparameter Optimization Study

To objectively select optimal training hyperparameters under severe class imbalance, an **Optuna Bayesian optimization study** was executed. The objective was set to maximize **Validation Macro-F1** rather than simple validation accuracy to ensure strong generalization on minority risk cases.

Trial #	Learning Rate	Dropout	Dense Units	Batch Size	Val Macro-F1	Training Time
Trial 0	0.000152	0.50	128	16	49.96%	214.3s
Trial 1	0.000485	0.30	128	16	24.95%	259.5s
Trial 2 (Best)	0.000554	0.50	256	16	53.51%	248.7s

4. Benchmark Performance & Comparative Model Evaluation

Each trained model was evaluated on the **same 1,043 unseen test images**. Standard statistical formulas were utilized: $Accuracy = (TP+TN)/(TP+TN+FP+FN)$, $Precision = TP/(TP+FP)$, $Recall = TP/(TP+FN)$, $Macro-F1 = (F1_Safe + F1_Mod + F1_High) / 3$. In road scene safety, **High-Risk Recall** and **Macro-F1** are prioritized over raw accuracy to prevent dangerous false negatives.

Model	Test Acc	Precision	Recall	F1-Score	Macro-F1	High-Risk Recall	Latency	Params
MobileNetV2 (Best)	80.82%	67.56%	73.11%	65.87%	65.87%	85.56%	11.3 ms	2.42M
EfficientNetB0	79.00%	65.45%	72.85%	64.79%	64.79%	79.97%	22.7 ms	4.21M
ResNet50	71.14%	66.51%	72.28%	60.58%	60.58%	72.20%	35.9 ms	23.85M
Custom CNN Baseline	68.55%	60.24%	55.21%	53.39%	53.39%	79.19%	5.4 ms	0.42M

Model Selection Rationale:

- 1. **Macro-F1 (65.87%)**: MobileNetV2 demonstrated the highest balanced metric across all three risk classes, successfully navigating the severe under-representation of Moderate Risk scenes.
- 2. **High-Risk Recall (85.56%)**: Correctly flagged **552 out of 645** unseen test high-risk road scenes. In accident prevention, missing a high-risk situation has catastrophic real-world consequences.
- 3. **Latency Efficiency (11.3 ms)**: Fast enough to sustain ~88 FPS on edge inference, outperforming EfficientNetB0 (22.7 ms) and ResNet50 (35.9 ms) while requiring only 2.42 million parameters.

5. Empirical Training Curves & Confusion Matrices

Below are the actual measured training histories and test set confusion matrices generated during evaluation.

A. MobileNetV2 (Selected Best Model)

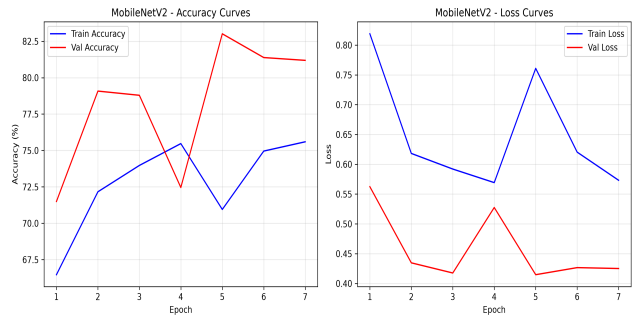


Figure 1a: MobileNetV2 Accuracy & Loss Curves

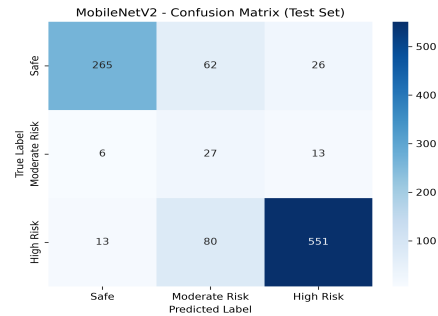


Figure 1b: MobileNetV2 Test Confusion Matrix

B. EfficientNetB0 (Runner-Up)

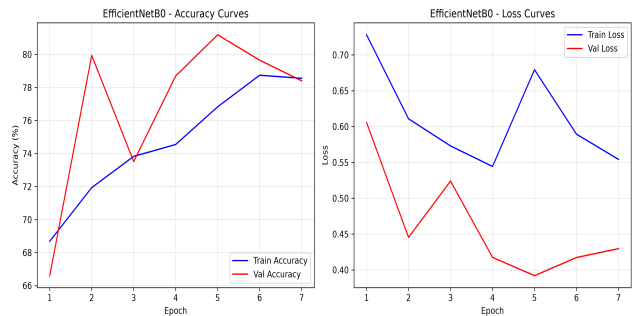


Figure 2a: EfficientNetB0 Accuracy & Loss Curves

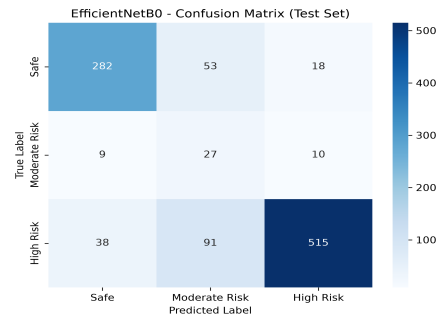


Figure 2b: EfficientNetB0 Test Confusion Matrix

C. ResNet50 & Custom CNN Baseline Benchmarks

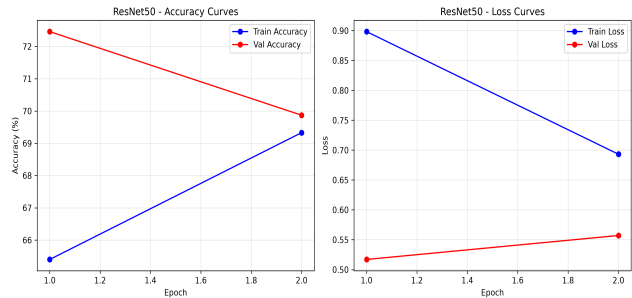


Figure 3a: ResNet50 Accuracy & Loss Curves

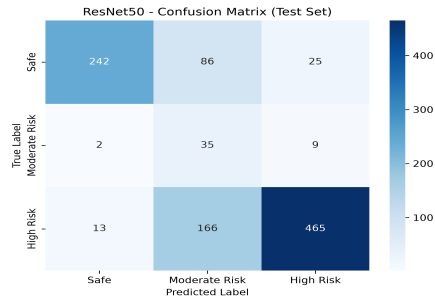


Figure 3b: ResNet50 Confusion Matrix

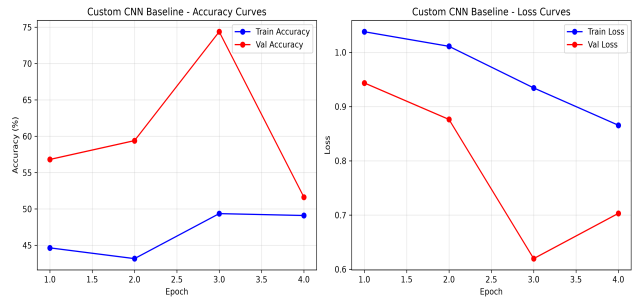


Figure 4a: Custom CNN Accuracy & Loss Curves

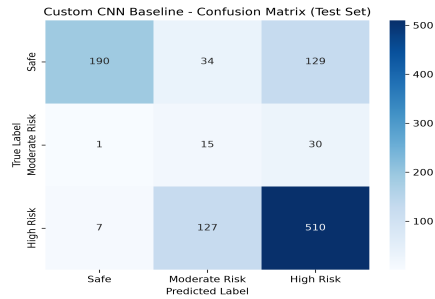


Figure 4b: Custom CNN Confusion Matrix

6. Full-Stack System Architecture & API Endpoints

The web application integrates the trained deep learning pipeline without modifying the frontend's visual design. The React UI communicates seamlessly with the Flask REST server running on port 5000.

HTTP Method & Route	Purpose / Functionality	Key Parameters / Payload	Return Data
POST /api/predict	Main inference endpoint for risk classification + YOLOv8 traffic analysis	Multipart form image + query ?model=best/cnn/mobilenet_v2/...	Risk, confidence, counts, density, safety recommendations, latency
GET /api/models/list	Supplies model dropdown in Prediction UI	None	List of models, training status, and active best model
GET /api/models/comparison	Supplies benchmark table in Analytics page	None	Measured metrics for all 4 models on unseen test set
GET /api/models/metrics	Supplies accuracy/loss curves and confusion matrix data	None	Historical epoch metrics and confusion matrix arrays
GET /api/predictions/stats	Feeds real-time dashboard cards	None	Total predictions, risk counts, traffic density breakdown
GET /api/health	Live backend connection status monitoring	None	{ status: 'online', version: '1.0.0' }

7. Operations & Execution Guide

The complete SafeRoad AI platform is fully automated and modular. To run or reproduce the system:

- 1. Launch Backend Server:** Run `npm run backend` or `py backend/app.py` (starts Flask API on port 5000).
- 2. Launch Frontend UI:** Run `npm run dev` (starts Vite development server on port 5173).
- 3. Access Interactive Prediction:** Navigate to `http://localhost:5173/prediction`. Upload any road scene image, choose a model from the dropdown (or use Best Model), and receive real-time classification, YOLOv8 detections, and hazard advisories.
- 4. Inspect Live Analytics:** Navigate to `http://localhost:5173/analytics` to compare all 4 models, inspect training curves, and analyze confusion matrices.
- 5. Retrain Models (Optional):** Run `py ml/train_cnn.py`, `py ml/train_mobilenet.py`, `py ml/train_efficientnet.py`, or `py ml/train_resnet.py` to execute standalone training pipelines.
- 6. Run Optuna Optimization:** Run `py ml/tune_models.py` to run Bayesian hyperparameter optimization on Validation Macro-F1.

8. Project Deliverables & Artifact Inventory

Directory / File	Description & Contents
<code>trained_models/mobilenetv2_best.keras</code>	Trained weights for Best Model (MobileNetV2, 80.82% acc, 23.8 MB)
<code>trained_models/efficientnetb0_best.keras</code>	Trained weights for EfficientNetB0 (79.00% acc, 30.7 MB)
<code>trained_models/resnet50_best.keras</code>	Trained weights for ResNet50 (71.14% acc, 98.2 MB)
<code>trained_models/cnn_best.keras</code>	Trained weights for Custom CNN Baseline (68.55% acc, 5.1 MB)
<code>results/model_comparison.csv & .json</code>	Comprehensive benchmark comparison across all 4 models on unseen test set
<code>results/optuna_tuning.json</code>	Optuna hyperparameter study records, trial trials, and optimal parameters
<code>results/{model}/confusion_matrix.png</code>	High-resolution test set confusion matrix heatmaps for each model
<code>results/{model}/training_curves.png</code>	Accuracy and loss trajectory plots across training and validation epochs
<code>backend/app.py & routes/ & services/</code>	Flask REST API backend with YOLOv8 and multi-model inference pipelines
<code>src/pages/Prediction.jsx, Analytics.jsx</code>	Integrated React UI components connected to live backend endpoints